

IAPR: Final project - Chocolate Recognition

Moodle group ID: 34

Kaggle challenge: *Deep learning*

Kaggle team name (exact): "Nom nom"

Author 1 (sciper): Aaron Kuriakose Dinesh (387804) **Author 2 (sciper):** Niyazi Gorkem Dener (370763)

Author 3 (sciper): Lunnet Newman Yokomizo (387120)

Due date: 21.05.2025 (11:59 pm)

Key Submission Guidelines:

- Before submitting your notebook, **rerun it from scratch!** Go to: `Kernel` > `Restart & Run All`
 - Only groups of three will be accepted, except in exceptional circumstances.
-

Table of Contents

Introduction

- Background: YOLOv8 Overview, Data and Preprocessing

Technical Description

- Model Architectures: YOLOv8Lite, Tiny YOLO, CompactYOLOv2
- Loss Definition
- Training

Model and Training Evaluation

- Qualitative Evaluation
- Quantitative Metrics

Bonus: Deformable DETR Model

Conclusion

Introduction

In this report, we present our work on detecting and classifying chocolates using deep learning. We explore three custom models inspired by different YOLO (You Only Look Once) architecture versions, each varying in complexity and design, as well as a bonus

transformer-based model explored briefly. We detail the architecture, design choices, and training procedures for each model, along with the technical justifications behind them. Finally, we evaluate their performance using object detection metrics and provide both quantitative results and qualitative analyses.

Background

YOLOv8 Overview

We based our model architecture on YOLOv8, an anchor-free object detection system developed by Ultralytics. As a preliminary validation step, we loaded a pretrained YOLOv8 model and evaluated it on our training set. Given its strong performance in this proof-of-concept test, we proceeded to adapt the architecture for our task.

YOLOv8 (You Only Look Once, version 8) is an object detector that performs both object localization and classification in a single forward pass. For each detected object, it outputs a tensor containing the normalized bounding box coordinates, an objectness score, and class probabilities over a predefined set of categories. Internally, the model processes the input image through a convolutional backbone to produce spatial feature maps. Each location on these maps corresponds to a cell in an implicit grid, and the model makes multiple predictions per grid cell to detect objects of various shapes and sizes.

The model is built on a convolutional neural network (CNN), which is well-suited for our dataset given its relatively limited size, especially compared to transformer-based architectures. Moreover, the deep layers of the YOLOv8 backbone enable hierarchical feature extraction across multiple spatial resolutions, which we expect allows the model to capture both the overall shape and finer surface textures of individual chocolates.

A key design feature of YOLOv8 is its anchor-free detection head, which simplifies training and enhances generalization by removing the need for manually designed anchor boxes. This allows the model to directly regress bounding box coordinates without relying on pre-defined priors, making it more flexible across diverse image conditions.

YOLOv8 consists of three main components:

- A CNN backbone responsible for initial feature extraction,
- A neck that fuses features across different spatial scales,
- And an anchor-free detection head that outputs bounding box coordinates, objectness scores, and class probabilities.

We implemented a model inspired by the YOLOv8 architecture for our final submission, called YOLOv8Lite. During our design explorations, we also implemented models we called CompactYOLOv2, and TinyYOLO, inspired by earlier YOLO versions.

References

Ultralytics YOLOv8 GitHub Repository: <https://github.com/ultralytics/ultralytics>

Ultralytics YOLOv8 Documentation: <https://docs.ultralytics.com/>

Ahmed, K., Yasir, A., & Rho, S. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. arXiv preprint arXiv:2304.00501. <https://arxiv.org/abs/2304.00501>

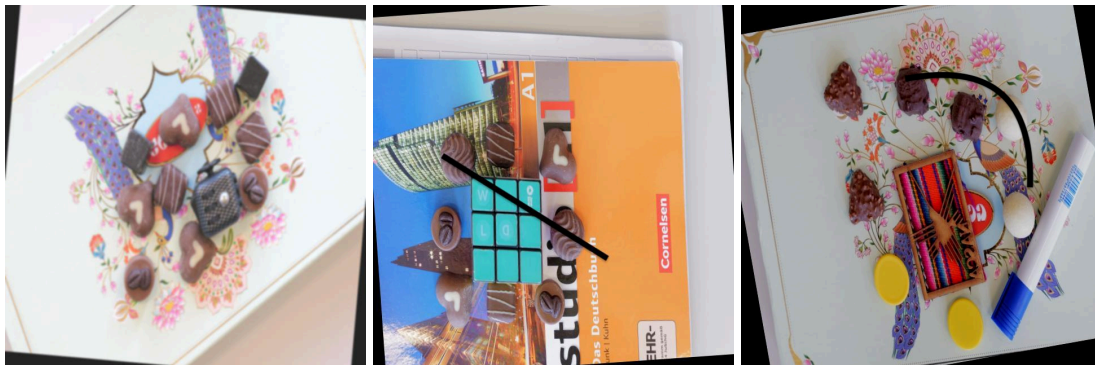
Data and Preprocessing

The training and testing datasets were annotated using Roboflow, where bounding boxes with class labels were added.

The training data were initially augmented using rotations between -15 and 15 degrees, horizontal and vertical flips, and brightness changes between -15 and +15%. In later experiments, additional augmentations were introduced, including Gaussian blurs, Gaussian noise, and manually added black occlusions to simulate the headband obstructions observed in some of the training and validation images (see examples below).

These occlusion augmentations were added after it was observed that the CNN model struggled particularly with segmenting and classifying chocolates obstructed by headbands. The goal was to provide the model with more representative examples to improve its robustness to such occlusions. Examples of images with black line occlusions are shown below. The remaining augmentations were intended to mimic natural variations that may occur in the dataset.

Following all augmentations, including on the reference images, the final augmented training set comprised 740 images.



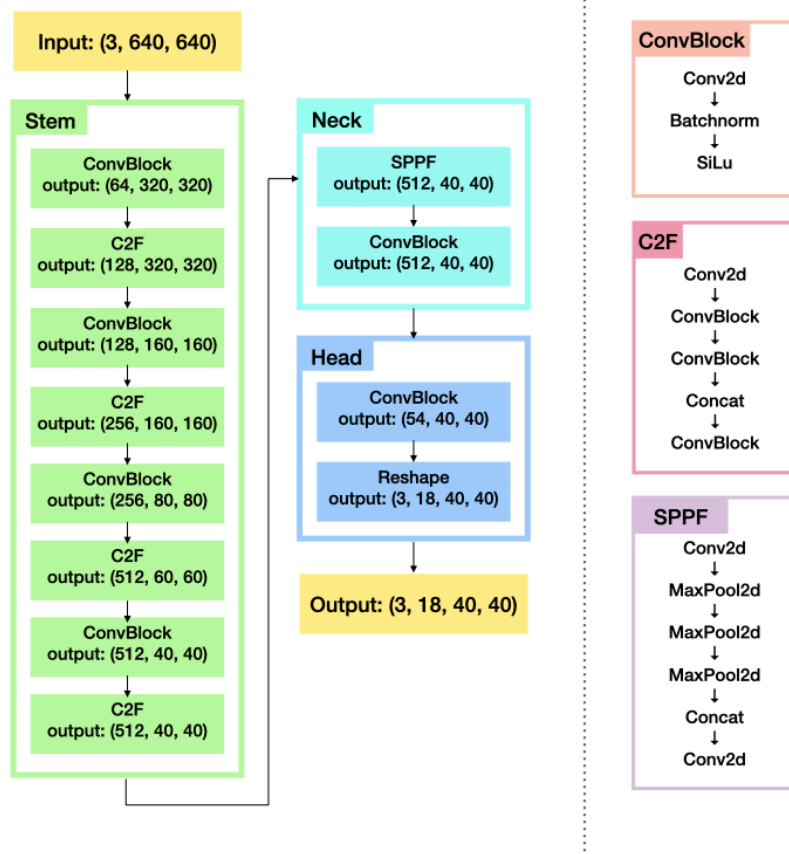
Examples of blurring, rotating, brightness changes, and adding black lines to the training images.

Technical Description

Model Architectures

YOLOv8Lite (Submission Model)

Below is a schematic of the YOLOv8Lite model, which is a lighter version of YOLOv8. Batchsize is taken to be 1.



The final output shape is $3 \times 18 \times 20 \times 20$, corresponding to 3 box predictions per spatial cell in the 20×20 grid. Each of the 18 features includes the 4 bounding box coordinates [cx (center x), cy (center y), w, h], an objectness score, and 13 class logits, totaling 18 values per prediction.

Much like YOLOv8, our model YOLOv8Lite has three main components: a Stem (CNN backbone), a Neck (for feature fusion), and an anchor-free Head (for detection). It also outputs the bounding boxes, objectness, and class scores for each image: (batch_size, anchors, 5 + num_classes, height, width). However, our head is single-scale (only one ConvBlock) while YOLOv8 has three.

ConvBlock is a building block for the CNN stem, which extracts the features by progressively downsampling the image while increasing channels, and is identical to YOLO v8's ConvModule.

The neck aggregates features using the SPPF module, which is a lightweight version YOLO v8's Spatial Pyramid Pooling Fast module's structure. Its purpose is to fuse multiscale information without increasing the feature map size.

The C2F Module is somewhat inspired by YOLOv8's C2f (Cross-Stage Partial with Feature concatenation) block in its general structure and feature concatenation, but is very simplified; for instance, we do not use skip connections. While in larger models this may cause some issues with vanishing gradients and earlier layers of the network not receiving a

strong gradient signal, we didn't find this to be an issue during our training. However, we do note that this might be a limitation when trying to scale our implementation.

The choice of SiLU activation was directly from YOLO v8, for which SiLU is the default. The SiLU activation function is one of the ways to obtain a continuous version of the ReLU activation. The SiLU is defined as follows:

$$\text{silu}(x) = x\sigma(x), \text{ where } \sigma(x) \text{ is the logistic sigmoid}$$

Overall, our model has the core components and principles of yolov8, but is single-scale, single-head and simplified. We choose to make these simplifications due to our limited dataset, relatively low number of classes and our limited compute capabilities. We prioritized a model that was fast to train on our limited hardware.

Total number of parameters of YoloV8Lite: 10,227,126. This model achieved a score on Kaggle of 0.93149, showing empirically that it works well.

Additional justifications for the choice of SiLU activations

The choice of SiLU over ReLU is a meaningful one. One known limitation of the ReLU activation function is its non-differentiability at $x = 0$, which necessitates the use of sub-gradients for optimization. However, it isn't clear which sub-gradient to choose since any gradient in the range $[0, 1]$ would be correct. Moreover, in non-convex optimization, sub-gradient methods often display poor performance.

Indeed, assume that we have some non-convex function f with the following conditions:

- There exists a constant $G > 0$ such that $\|g\|_2 \leq G \quad \forall g \in \partial f(x)$ where ∂f is a sub-gradient of f
- The distance between the initialization x^0 and the optimal point x^* is bounded:
 $\|x^0 - x^*\|_2 \leq R$
- We use the step-size:

$$\alpha_k = \frac{R}{G\sqrt{k}}$$

Then, the iterates generated by the sub-gradient method satisfy:

$$\min_{0 \leq i \leq k} f(x^i) - f(x^*) \leq \frac{RG}{\sqrt{k}}$$

Note that condition (1) is automatically satisfied when f is G -Lipschitz. In practice, however, this exact convergence rate cannot be achieved since we cannot compute R and it is computationally unfeasible to compute G . Consequently, we can only hope for a suboptimal convergence rate of $\mathcal{O}(1/\sqrt{k})$ when using sub-gradient methods, where k is the number of iterations.

Additionally, when using stochastic subgradient methods, we have the following convergence in expectation guarantees:

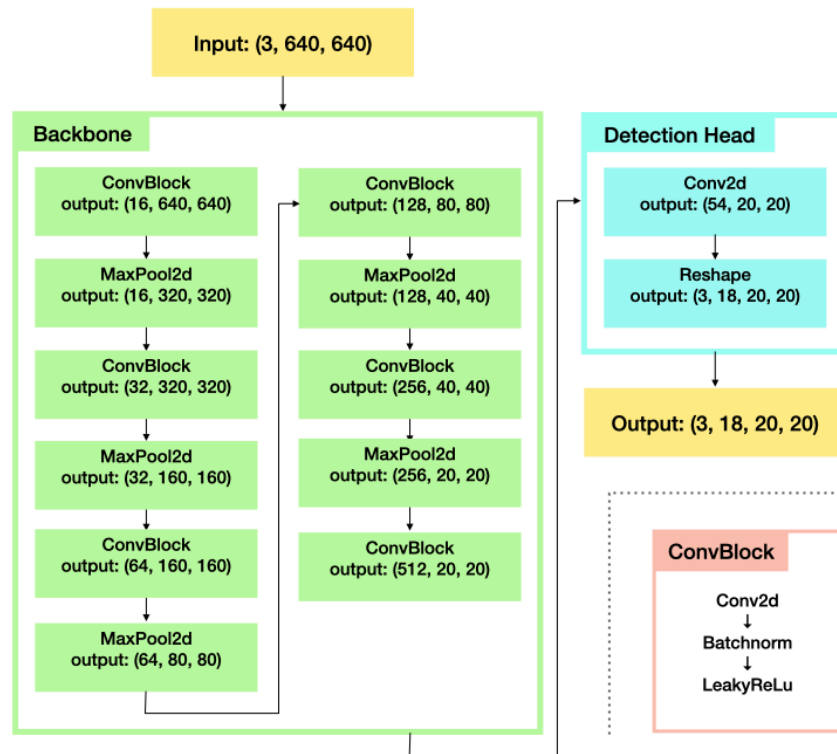
$$\mathbb{E} [f(x^k) - f(x^*)] \leq \left(\frac{D^2}{\gamma_0} + \gamma_0 M^2 \right) \frac{2 + \log k}{\sqrt{k}}$$

where D , M , and γ_0 are related to the problem geometry and step size.

These convergence rates are relatively slow. In contrast, when working with a fully differentiable function, it is possible to achieve a $\mathcal{O}(\frac{1}{k})$ convergence rate. Hence, it is in our best interest to have a fully differentiable activation function. The Ultralytics team also state that the non-monotonicity of the SiLU function might help the model learn more complex patterns. Also since the gradient of the SiLU function is non-zero near $x = 0$ it can help avoid vanishing gradients in the case where the pre-activation outputs are normalized or are close to 0.

Tiny YOLO

Before testing our final submission model, YOLOv8Lite, we implemented TinyYOLO, a lightweight object detector inspired by the original YOLO architecture. It consists of a purely sequential convolutional backbone with interleaved max-pooling layers that progressively reduce the input resolution from 640×640 to a 20×20 spatial grid. A final 1×1 convolutional layer serves as the detection head. At each grid cell, the model predicts 3 bounding boxes, each with associated class probabilities and objectness scores. Below is a schematic of the Tiny YOLO model, where batchsize is taken to be 1.



Again, the final output shape is $3 \times 18 \times 20 \times 20$, corresponding to 3 box predictions per spatial cell in the 20×20 grid. Each of the 18 features includes the 4 bounding box coordinates [cx

(center x),cy (center y),w,h], an objectness score, and 13 class logits, totaling 18 values per prediction.

Unlike YOLOv8Lite, TinyYOLO does not follow a stem–neck–head design. It also uses an anchor-based prediction strategy, meaning that each bounding box prediction is interpreted relative to a predefined anchor configuration.

We chose the Leaky ReLU activation function, consistent with the original YOLOv1–v3 implementations. Leaky ReLU is a modification of the standard ReLU that allows a small, non-zero gradient when the input is negative. It is defined as:

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

where $\alpha \in (0, 1)$ is a small constant that controls the slope for negative values. In both the original YOLO models and our implementation, we set $\alpha = 0.1$.

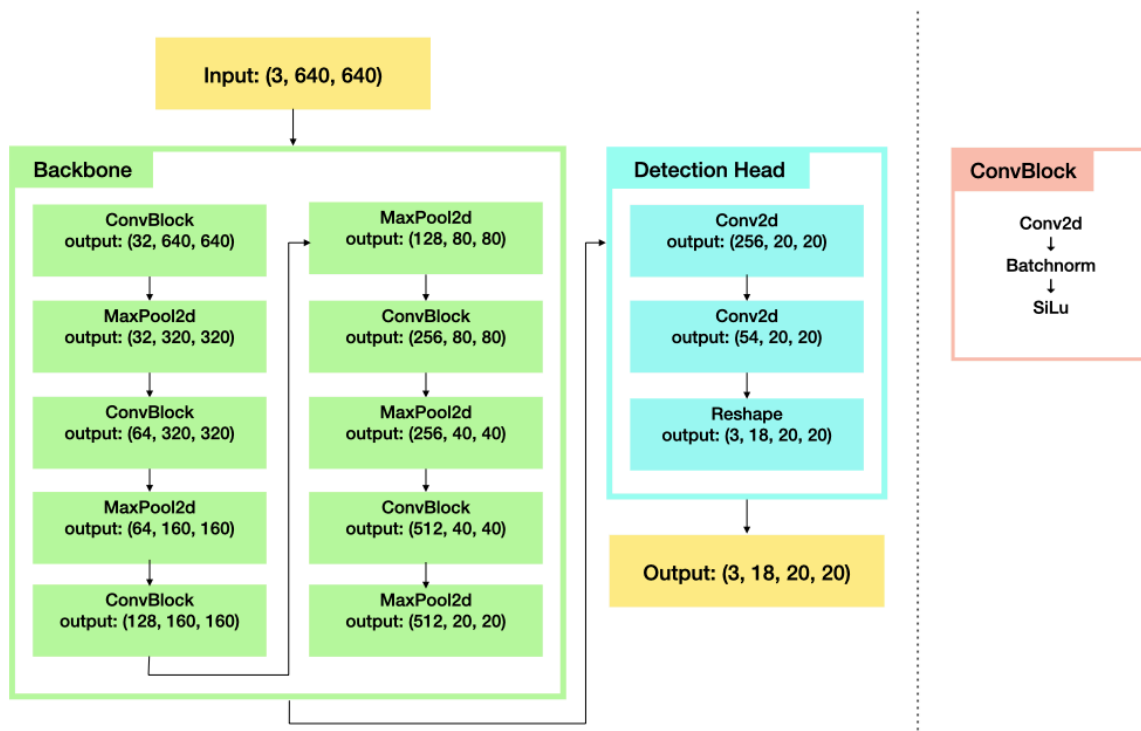
Total number of parameters of TinyYolo: 1,602,486

References:

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788. [arXiv:1506.02640](https://arxiv.org/abs/1506.02640)

CompactYOLOv2

As part of our design exploration, we also implemented CompactYOLOv2, a lightweight object detection model that serves as a midpoint in complexity between TinyYOLO and YOLOv8Lite. This model retains a simple, fully convolutional structure but incorporates SiLU activations and a two-stage detection head. Below is a schematic of the CompactYOLOv2 model, where batchsize is taken to be 1.



The backbone consists of five convolutional blocks, each followed by batch normalization, a SiLU activation, and a max-pooling layer that halves the spatial resolution. Compared to TinyYOLO, this architecture uses higher initial channel depth and smoother activations (SiLU instead of Leaky ReLU), which may lead to better convergence.

The detection head is a shallow two-layer module, while Tiny Yolo had one.

Like TinyYOLO, CompactYOLOv2 is anchor-based and performs detection at a single scale. However, the use of SiLU activations (as in YOLOv5 and YOLOv8) and a more expressive head may lead to better results.

Total number of parameters of CompactYolo: 2,764,854

Loss Definition

To optimize our YOLO-inspired models, we implemented a loss function structurally similar to the one used in the Ultralytics YOLOv8 codebase. Like the original, our loss comprises three components: a bounding box regression loss, an objectness loss, and a classification loss. However, our implementation is simplified in several ways:

- **Coordinate loss (box regression):** We compute the mean squared error (MSE) between the predicted and target box coordinates, restricted to positions where an object is present. In contrast, the original YOLOv8 loss uses Complete IoU (CIoU) loss, which penalizes not only misalignment but also poor overlap, distance between box centers, and mismatched aspect ratios. Our simplified choice of MSE does not encourage large or tightly fitting boxes; as a result, predicted bounding boxes are small and do not cover

the full extent of each chocolate. However, this appears not to affect classification performance, which is the primary goal of our task. Since finding the box area is not required, MSE proved sufficient for training.

- Objectness loss: This component predicts the confidence that an object exists in each proposed bounding box. We use binary cross-entropy (BCE) with logits, which is numerically stable and consistent with the original YOLOv8 implementation.
- Classification loss: This penalizes incorrect class predictions for locations containing an object. Like the objectness term, it uses BCE with logits, allowing for multi-class probability estimation.

Binary Cross Entropy (BCE) loss is defined as:

$$\mathcal{L}_{\text{BCE}}(p, y) = -[y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

Here:

- $p \in (0, 1)$ is the predicted probability that the class is 1 (typically output by a sigmoid function),
- $y \in \{0, 1\}$ is the true binary label.

BCE loss penalizes confident incorrect predictions more than uncertain ones. In practice, the "BCE with logits" function is often used, which combines the sigmoid activation and BCE loss in a numerically stable way. This is useful when raw logits (not probabilities) are directly passed to the loss function.

The final loss is computed as the unweighted sum of the three components, whereas the original YOLOv8 loss applies scaling factors to balance their contributions.

We chose to make these simplifications to make the reduce implementation complexity and focus on the core model architecture. Additionally, given the limited size and scope of our dataset, we found that a simplified loss was sufficient to achieve good performance. Simpler loss terms also enabled faster training iterations and more interpretable error analysis.

Training

All models were trained using the PyTorch framework. We used the Adam optimizer with a fixed learning rate of 10^{-4} . The training procedure lasted 200 epochs, and models were evaluated at each epoch using the YOLO-style loss described above. During training, we monitored training loss, test loss, mAP@IoU on both training and test sets, and the F1 score computed after applying non-maximum suppression. The model with the best F1 score after epoch 80 was saved separately as the final submission model. Checkpoints were saved every N epochs, and the best-performing models based on training loss and F1 score were retained.

Rather than splitting our training data further, we used the provided Kaggle test set as our validation set, which we annotated ourselves. This decision allowed us to maximize the amount of data available for training, which is important considering our small dataset. Although this introduces a potential source of human annotation error, we judged that the

benefits of improved generalization from a larger training set outweighed that risk. Additionally, we did check that we did not make any annotation mistakes.

Model and Training Evaluation

Qualitative Evaluation

We evaluated the model's performance after training on data with augmentations.

Comparison of Different Models



Image 1 (left) shows the object detection results with the TinyYOLO model. There is one wrong detection, one undetected object, two correct detections that are missclassified and three correct detections with correct classification.

Image 2 (center) shows the object detection results with the CompactYolov2 model. There is one undetected object, one correct detection that is missclassified and four correct detections with correct classification.

Image 3 (right) shows the object detection results with the YOLOv8Lite model, our submission model. There is one undetected object, one correct detection with missclassification and four correct detections with correct classification.

Based on the example above, one can conclude that TinyYOLO is insufficient for the task. CompactYOLO and YOLOv8Lite, by contrast, appear to produce identical results on this particular image. This raises a natural question: why not prefer the simpler model with fewer parameters if the outcome appears similar?

However, this interpretation is misleading. The similarity in output occurs in just one image; overall, YOLOv8Lite consistently produces better results across the dataset. For instance, in a more detailed analysis of the two models, YOLOv8Lite detects and correctly classifies the Triangolo chocolate with a confidence score of 0.99, whereas CompactYOLO detects the same object with only 0.08 confidence.

All models were trained for 200 epochs, and we report results using the version that achieved the lowest training loss. Additionally, non-maximum suppression was applied during all inference runs to filter overlapping predictions.

Training Improvements over Epochs for YOLOv8Lite



All three images are with the YOLOv8Lite model, trained over different epochs.

Image 1 is trained with 40 epochs. There are four wrong detections, one undetected object, three one undetected object and two correct detections with correct classification.

Image 2 is trained with 120 epochs. There is one wrong detection, one undetected object, one correctly detected but misclassified object and four correct detections with correct classification.

Image 1 is trained with 200 epochs. There is one undetected object and five correct detections with correct classification.

From these results it can clearly be concluded that training improves detection and classification qualitatively.

General Observations

The model performed well overall, producing high-confidence predictions (typically between 0.8 and 1.0) for most chocolate types. Occasionally, for challenging images, the prediction confidences can be as low as 0.2. Bounding boxes were generally accurate and well-placed, though they often did not cover the full extent of the chocolates. This partial coverage did not appear to negatively affect classification performance.

Prediction (Filtered): L1000902.JPG.rf.a61e76f2699241af8bd8adb8c3e593c0.jpg



Background Influence

The model handled patterned and cluttered backgrounds surprisingly well, provided the chocolates were not obscured.

However, highly distracting backgrounds, such as the cover of textbooks, led to occasional missed detections or false positives, particularly for chocolates with low contrast against the background. For instance, Jelly Milk was sometimes falsely detected in background regions with similar color.

Overall, performance declined when chocolates blended into the background, highlighting the importance of contrast for effective segmentation. Small false positives also occurred when background colors resembled certain chocolates.

Prediction (Filtered): L1000857_JPG.rf.a3a00f35068593164d0bad0df1569ec8.jpg

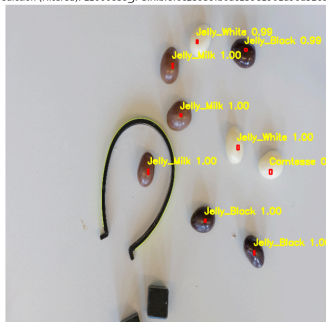


Occlusion and Object Interference

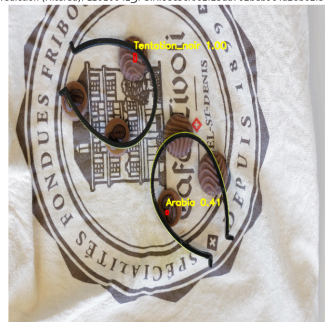
Black headbands did not impair detection as long as they did not obscure the chocolates. However, when obstructions covered part of a chocolate, missed detections became common.

Similarly, the presence of additional objects in the image did not affect performance unless they overlapped with the chocolates or cast strong shadows over them.

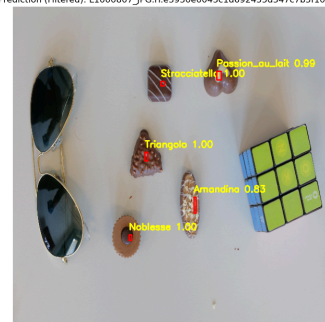
Prediction (Filtered): L1000839_JPG.rf.bf8f6c23959fb0ac139e1901a96d32cc.jpg



Prediction (Filtered): L1010042_JPG.rf.88c3cf662f25aa702beb9c4a28b62f5a.jpg



Prediction (Filtered): L1000807_JPG.rf.a3930e0043c1d692455d347c7b5f10ef.jpg



Per-Class Performance

Amandinas were the most challenging: confidence scores sometimes dropped to ~0.2, likely due to their complex visual features—fine patterns and multiple colors.

Jelly Whites were occasionally missed when heavily shadowed.

In contrast, the model successfully distinguished visually similar pairs:

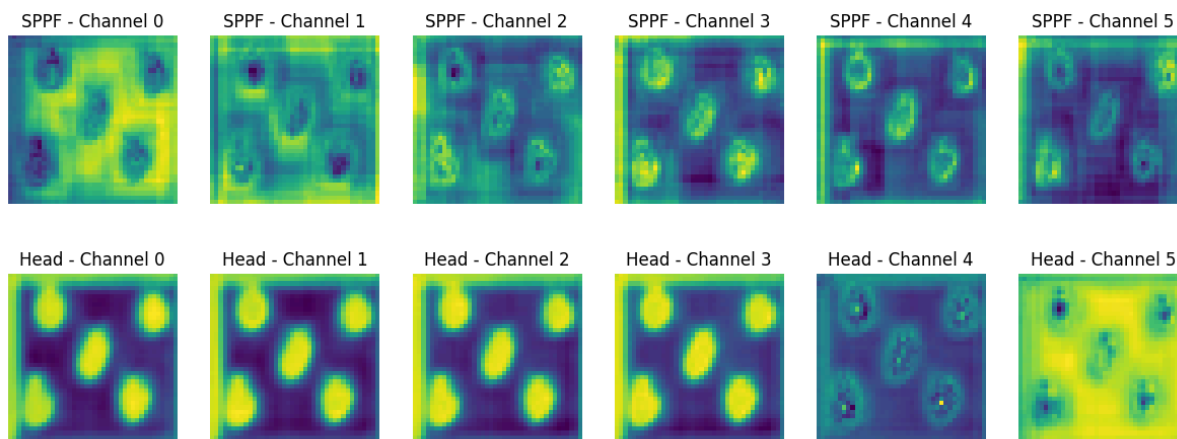
- Comtesse vs. Jelly White, suggesting strong shape recognition.
- Triangolo vs. Tentation Noir, indicating the model learned subtle differences in shape and color despite shared textures.

Features Extracted

As stated above, we suspected that the extra features extracted from noisy backgrounds might be causing the model to miss chocolates or to miss-classify certain parts of the background as chocolates. To test this hypothesis, we can visualize the feature maps for inputs at different points in the network, namely the SPPF module and the Detection heads.

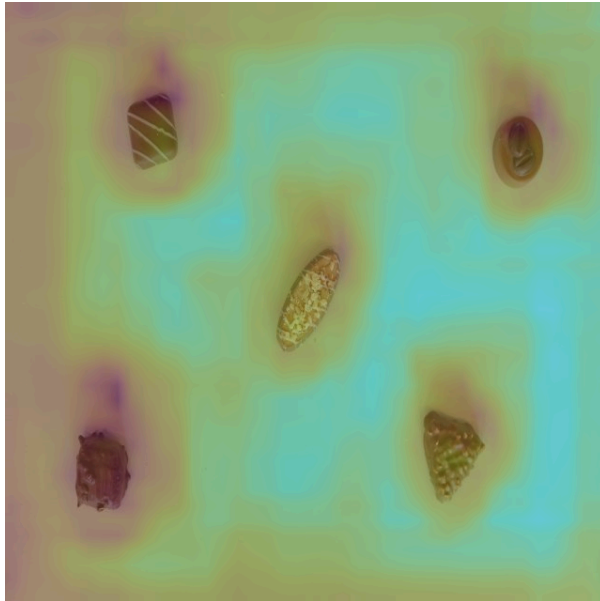


This is the first image that we passed through the model. As can be seen, this should be an easy task for the model since the chocolates are clearly visible with the background being plain. Below are the feature maps from the SPPF module and the Detection head:



We can see that the SPPF module aggregates the features from multiple scales in order to localize the regions where objects might lie. The detect head then performs a rudimentary segmentation and classification of the object. It is evident that on a plain background such

as the table, our model has no problem in localizing and classifying the object. This is further reinforced by looking at an overlay of the feature map on the original image:

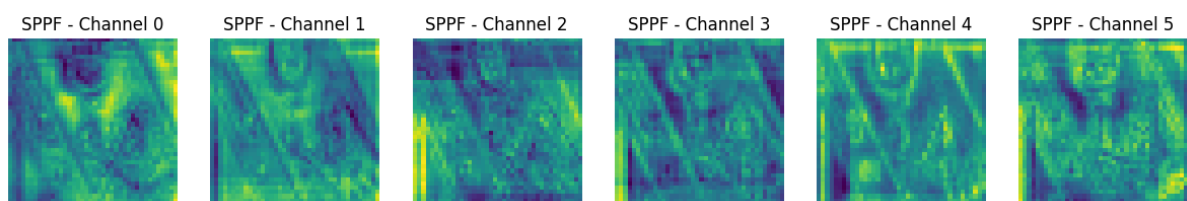


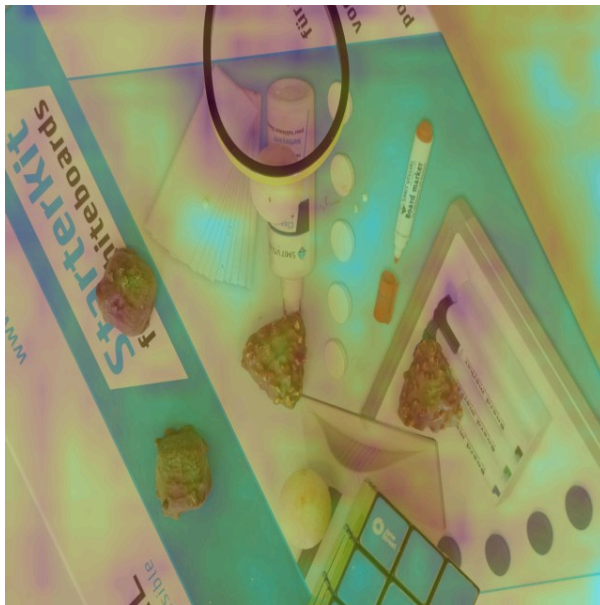
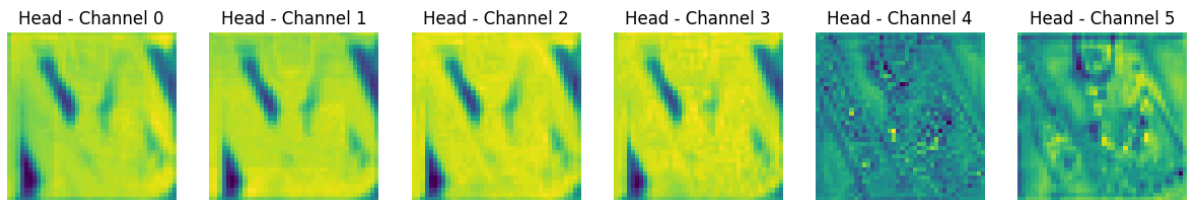
We can see that the model focusses well on the chocolates (indicated by the reddish color above the chocolates).

For the second input we decided to choose an image with a more complex background.



The activations of the SPPF and the Detection heads, respectively, can be seen below:



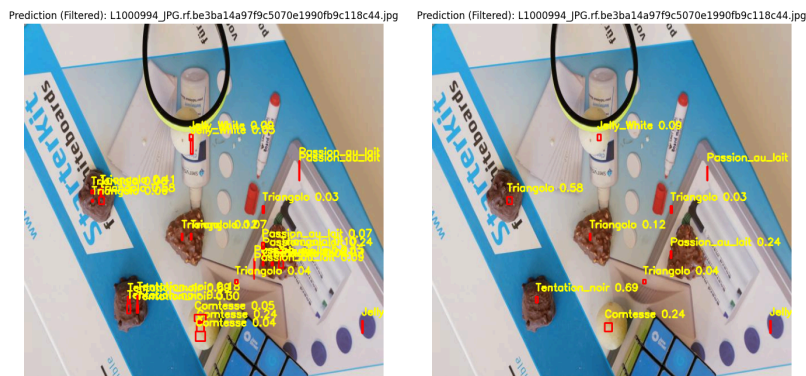


When combining this with the overlaid feature map we can see that the presence of a complex background tends to confuse the model with regards to the localization of the chocolates. We also noticed a drop in the classification accuracy when comparing images that had a simple background versus those with a complex background. This again further reinforces the idea that our model has area to improve when dealing with images with more complex backgrounds.

Quantitative Metrics

Non-Maximum Suppression Effect

The raw model output alone is insufficient, as it often includes multiple overlapping predictions for the same object. Non-Maximum Suppression (NMS) is essential for eliminating redundant detections, particularly false positives that repeatedly identify the same object within its surrounding region.

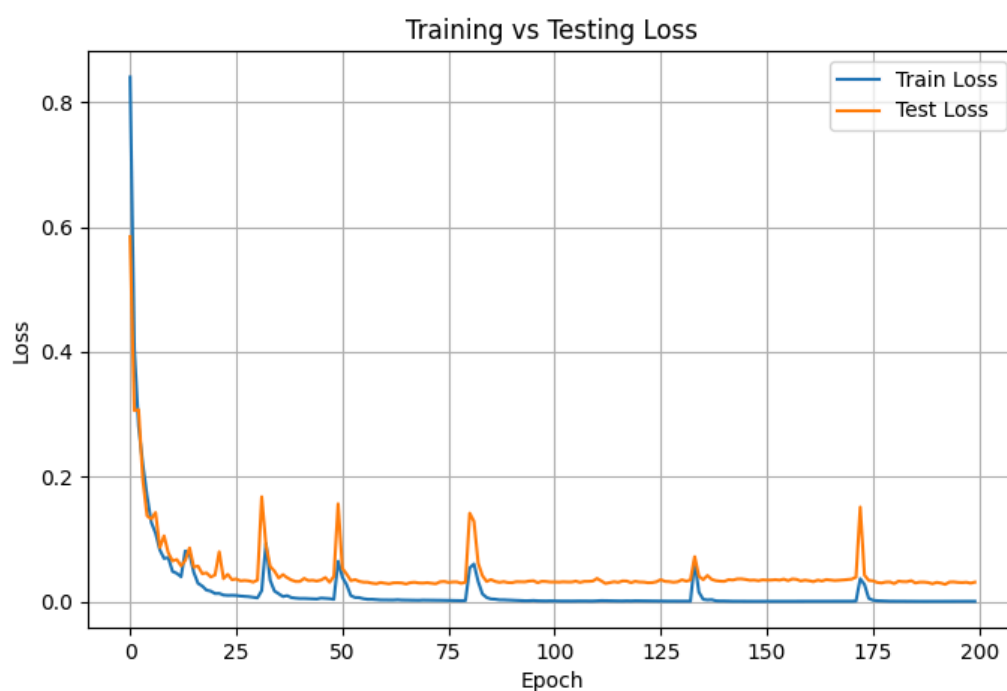


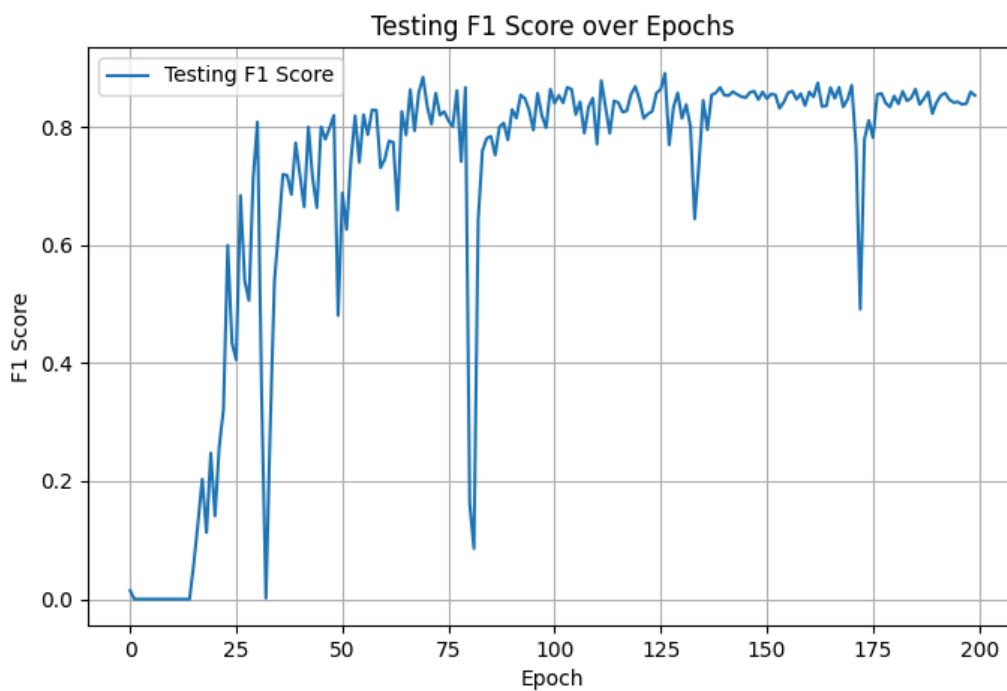
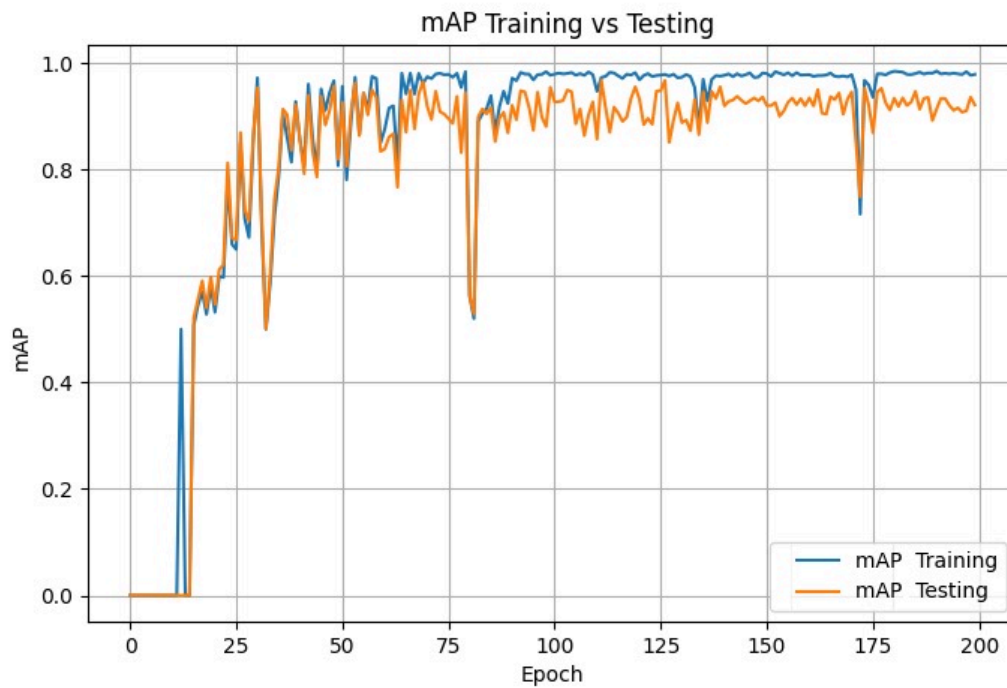
The image on the left shows the raw detection outputs without non-maximum suppression (NMS), while the image on the right shows the result with NMS applied, using the YOLOv8Lite model trained for 40 epochs. In the first image, multiple overlapping bounding boxes are predicted for the same object. In contrast, the second image retains only the highest-confidence box for each object, effectively eliminating redundant predictions within a predefined spatial neighborhood.

The NMS algorithm is straightforward. After generating the set of predicted bounding boxes and their associated confidence scores, the algorithm iterates through the list. For each box, it computes the Euclidean distance to other boxes in the same vicinity. Among overlapping or nearby boxes, only the one with the highest confidence score is retained; all others are suppressed. This process ensures that each object is represented by a single, most confident detection.

Training metrics

The loss, mAP@0.0001, and f1 curves for YOLOv8Lite are shown below.





The model demonstrated good training properties with our loss decreasing to its minimum in a manner expected of stochastic methods such as Adam. It also showcased good performance on both the training set and the test set: as we can see in the graphs above, the training and test loss curves follow each other quite closely diverging only slightly towards the end of training. This is indicative of minimal overfitting in our model.

We wish to clarify the specifics of our implementation of the mean Average Precision (mAP) metric. In our experiments, we computed mAP at a very low Intersection over Union (IoU) threshold of 0.0001.

Mean Average Precision at a given IoU threshold (denoted $\text{mAP}@k$) is a metric for evaluating object detection models. It reflects both classification and localization performance. The procedure is as follows:

To compute $\text{mAP}@k$:

- For each predicted bounding box, the Intersection over Union (IoU) with the ground truth is calculated:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

- A prediction is considered correct if $\text{IoU} \geq k$ and the predicted class is correct.
- Precision–Recall curves are computed per class, and the area under each curve (Average Precision, or AP) is calculated.
- $\text{mAP}@k$ is the mean of AP across all classes:

$$\text{mAP}@k = \frac{1}{C} \sum_{c=1}^C \text{AP}_c, \text{ where } C \text{ is the total number of classes.}$$

In our case, the bounding box predictions tend to be very small due to the simplifications made in the loss function, particularly the use of MSE instead of IoU-based loss. This causes the overlap areas (used in the IoU calculation) to be negligible, making standard IoU thresholds inappropriate. To account for this, we set the IoU threshold to a very low value (0.0001), effectively treating a box as correct if it is correctly located, regardless of its exact size.

This makes our $\text{mAP}@0.0001$ implementation more of a localization metric than a strict detection quality measure.

The non-smoothness of the mAP curve indicates that our model parameters may be cycling around the optimum, without actually converging. This is common with stochastic methods with fixed learning rates as we never evaluate the full gradient and can overshoot the minimum. To fix this we likely could make use of a learning rate scheduler to monotonically decrease the learning rate during training. This would help us to slowly converge to the minimum.

Bonus Exploration: Simplified Deformable DETR Model

In addition to our YOLO-inspired architectures, we briefly explored a transformer-based approach using Deformable DETR (Deformable DETection TRansformer, DDETR). Deformable DETR is a variant of the original DETR, a transformer-based architecture, with an added multi-scale deformable attention, although we use a single-scale deformable attention in our implementation. This mechanism allows the model to attend to a sparse set of key sampling points around each object, rather than attending to the entire image grid.

Deformable Attention Module Overview

In standard attention mechanisms, each query vector q_i attends to all key vectors k_j in the sequence. This requires computing attention scores through dot products between every query–key pair:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

This operation scales quadratically with the input size, which is too expensive for computer-vision tasks using high-resolution images. Additionally, because attention is distributed uniformly over all positions, detecting small objects often requires extensive training, making convergence slow.

Deformable attention addresses this limitation through two key ideas:

- Sparse, learnable sampling:
 - Instead of attending to all keys, each query $\mathbf{z}_q$ attends to a small, fixed number K of sampling locations.
 - These locations are predicted as learnable offsets $\Delta\mathbf{p}_{mqk} \in \mathbb{R}^2$, relative to a reference point $\mathbf{p}_q$, where m indexes the attention head and k indexes the sampling point.
 - Over successive layers, the attention focuses on relevant object regions, enabling localized and efficient feature aggregation.
- No explicit keys:
 - Unlike traditional attention, deformable attention does not use key projections.
 - For each query $\mathbf{z}_q$, the model directly predicts:
 - Sampling offsets $\Delta\mathbf{p}_{mqk}$ and
 - Corresponding normalized attention weights $A_{mqk} \in [0, 1]$ with $\sum_{m=1}^M \sum_{k=1}^K A_{mqk} = 1$.

The single-scale deformable attention output is computed as:

$$\text{DeformAttn}(\mathbf{z}_q, \mathbf{p}_q, \mathbf{x}) = \sum_{m=1}^M \mathbf{W}_m \left[\sum_{k=1}^K A_{mqk} \cdot \mathbf{W}'_m \cdot \mathbf{x}(\mathbf{p}_q + \Delta\mathbf{p}_{mqk}) \right]$$

Where:

- $\mathbf{z}_q$ is the query embedding for the q -th position,
- $\mathbf{p}_q$ is its associated reference point,
- $\Delta\mathbf{p}_{mqk}$ is the learnable offset,
- A_{mqk} is the attention weight for the k -th sampling point in head m ,
- $\mathbf{x}(\cdot)$ is the bilinear interpolated feature at the sampled location,
- $\mathbf{W}_m$ and $\mathbf{W}'_m$ are learnable linear projection matrices for output and value embeddings,
- M is the number of attention heads,
- K is the number of sampling points per head.

This design achieves the following benefits:

- The computational cost is reduced per query.
- The attention mechanism becomes spatially focused.

We were motivated to test Deformable DETR because its sparse attention mechanism aligns well with our goal: detecting and classifying chocolates by focusing on local features (i.e., the chocolate and its immediate surroundings), while ignoring large, distracting background regions. We also wondered whether Deformable DETR's attention could handle small and detailed features more effectively than the CNN-based YOLO-inspired models. While we were aware that transformer-based models require large datasets, we hoped that our data augmentation pipeline would mitigate this, and we aimed to explore the model's potential on our task.

In our setting, we implemented a simplified version of Deformable DETR and trained it on the same augmented chocolate detection dataset.

References Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). Deformable DETR: Deformable Transformers for End-to-End Object Detection. arXiv preprint arXiv:2010.04159. Available at: <https://arxiv.org/abs/2010.04159> GitHub: <https://github.com/fundamentalvision/Deformable-DETR>

Overview of Model Components - probably delete

Backbone: we use a lightweight convolutional neural network (CNN) extracts feature maps from the input image, downsampling the spatial dimensions by a factor of 8. We trained two different backbones, both CNNs, of different complexities.

Positional Encoding: we used standard sinusoidal positional encodings.

Transformer: uses single-scale deformable attention (MSDeformAttn) to sample informative features from a sparse set of reference locations. For each query, the model predicts a sparse set of sampling locations and aggregates features from those points using bilinear interpolation. The transformer progressively refines object queries across layers and is responsible for both localization and classification.

Detection Heads: one outputs class logit predictions, while another outputs normalized box coordinates.

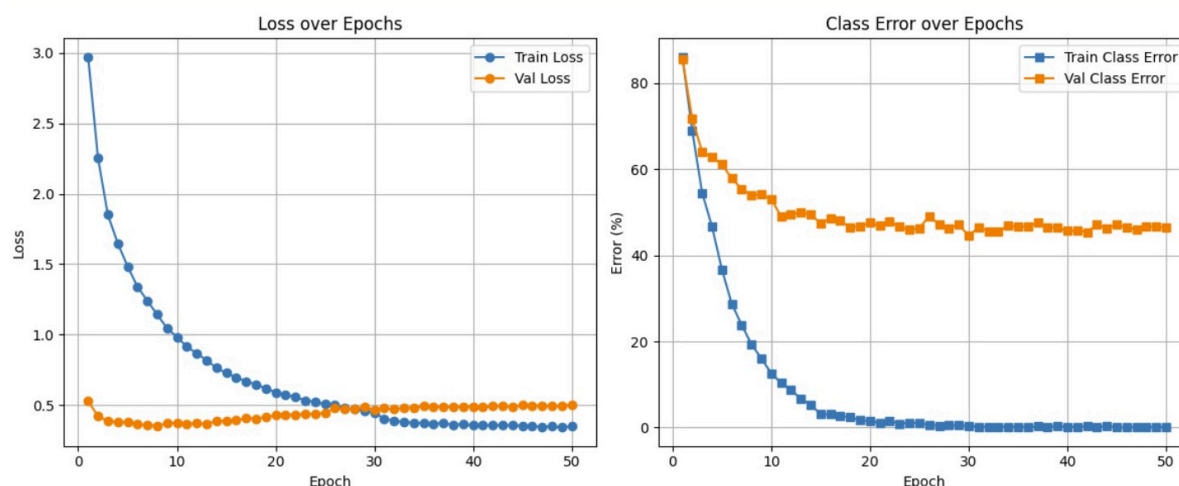
Loss and Matching:

- The loss function is almost identical to the reference's and uses an algorithm called Hungarian matching to align predictions with ground truth in a one-to-one manner.
- The loss includes a classification loss, a box coordinate loss, and a generalized IoU (GIoU) loss to improve box quality and overlap.

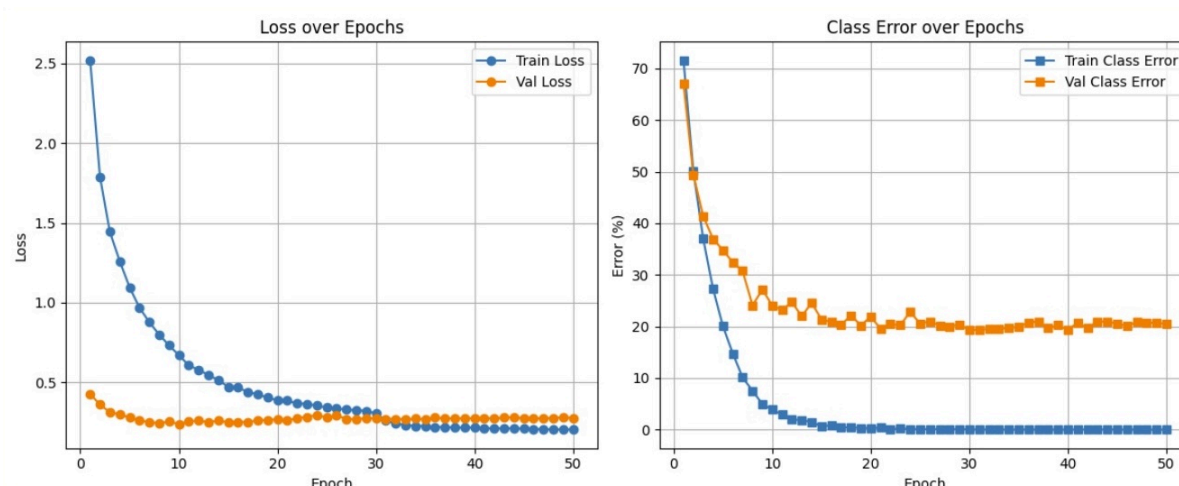
In theory, this architecture is a powerful model for detection tasks requiring flexibility in spatial attention. It should be able to better handle diverse backgrounds and subtle features.

Performance

Below we present the training and validation curves for our simplified Deformable DETR model under two different configurations.



In the first experiment, we used a relatively complex CNN backbone resulting in a model with approximately 8.5 million parameters. As shown above, the model begins to overfit early in training: validation loss increases after 5 epochs and surpasses the training loss after around 27 epochs. Furthermore, the validation classification error plateaus at around 50%, which is high and suggests poor generalization.



To lessen overfitting, we then reduced the backbone complexity, lowering the total parameter count to approximately 2.2 million. This led to a small improvement in classification performance (validation error plateau-ed at around 20%) but the model continued to overfit.

These results suggest that the Deformable DETR model, even in its simplified form, is not well-suited to our dataset. Despite downscaling the model, overfitting remains an issue due to the limited size our training data. This reinforced our choice of using CNN-based architectures like YOLOv8Lite, which showed better generalization.

Conclusion

Summary

This report presents a deep learning pipeline for the detection and classification of various types of chocolates. Our work centers on the development and evaluation of three custom object detection models inspired by the YOLO family: TinyYOLO, CompactYOLOv2, and a deeper, anchor-free architecture called YOLOv8Lite, which serves as our final submission model.

Through both qualitative and quantitative evaluations, we found that YOLOv8Lite outperformed TinyYOLO and CompactYOLOv2, particularly in terms of detection confidence and robustness. We conducted experiments to determine the optimal number of training epochs, and observed that model performance improved with sufficient training. Additionally, we analyzed how background variations affect detection quality, and attempted to interpret the internal feature representations to explain performance differences across models.

Finally, we briefly explored a transformer-based detection approach using Deformable DETR, providing a point of comparison with our CNN-based architectures.

Further work

From the results of our experiments, we observed that deep learning models with a high number of parameters, when trained sufficiently, tend to yield strong performance. However, the background of the images can still confuse the model during feature extraction, potentially leading to a drop in accuracy or F1 score. We outline two directions that could help improve performance.

Increasing the Input Image Resolution

The original images provided were 6000×4000 pixels, but we downsampled them to 640×640 for training. This aggressive reduction likely causes significant information loss, especially in fine-grained textures or shape details. Increasing the input resolution could preserve more discriminative features, potentially leading to improved detection and classification accuracy.

Using Multiple Models Based on Background Type

An alternative approach we considered but did not implement due to time constraints involves training specialized models for different background types. Specifically, we propose training seven separate models, six of which are dedicated to images with a specific background. To achieve this, one of them would be an initial lightweight classifier that would first predict the background type of a given input image. During inference, this classifier would route the image to one of the six corresponding specialized model.

This architecture (one background classifier plus six specialized detectors) could enable each model to focus on learning features most relevant to its assigned background, potentially improving detection and classification performance. The total parameter count across all models could be constrained to remain under 12 million, preserving a manageable computational footprint.

